

An AutoML-based approach for Network Intrusion Detection

Nana Kankam Gyimah*, Robert Akinie⁺, Judith Mwakalonge*,
Benibo Izison*, Arthur Mukwaya*, Denis Ruganuzi*, Methusela Sulle*

* South Carolina State University, Orangeburg, South Carolina, US, 29117

⁺ North Carolina A&T State University, Greensboro, North Carolina, US, 27411

Abstract—Network intrusion detection (NID) is a crucial cybersecurity measure for detecting unauthorized access and malicious activity. Traditional signature-based detection is highly accurate for known threats but struggles with zero-day attacks, requires frequent updates, and suffers performance issues as its database grows. Machine learning (ML) improves NID by identifying both known and unknown threats without constant updates, adapting to evolving cyber risks. However, traditional ML methods rely on manual processes, making them complex, less efficient for large-scale data, and prone to lower accuracy and false alarms. In this paper, we evaluate AutoML frameworks for network intrusion detection, developing automated algorithms to enhance performance, reduce false positives, and eliminate manual feature selection, while comparing them to traditional machine learning using the NSL-KDD dataset. Experiments on the NSL-KDD dataset demonstrate that while gradient boosting models (XGBoost, CatBoost) surpass bagging-based methods (Random Forest, LightGBM) with 80% accuracy, AutoML frameworks further improve detection capabilities through automated hyperparameter optimization and model selection, with H2O AutoML and MLJar achieving the highest accuracy of 90%, significantly outperforming FLAML (79%) and traditional models. These findings underscore AutoML's potential to optimize intrusion detection, improve detection robustness, and provide scalable, adaptive cybersecurity solutions.

Index Terms - Network Intrusion Detection (NID), Automated Machine Learning (AutoML), Cybersecurity Threat Detection, Hyperparameter Optimization

I. INTRODUCTION

Network intrusion detection (NID) is a crucial aspect of cybersecurity, aimed at identifying unauthorized access or malicious activity within a network [1]. Its importance has grown significantly as networks expand and become increasingly complex, with the potential consequences of security breaches becoming more severe. According to the Special Report on Cyberwarfare in the C-Suite, Cybersecurity Ventures projects that global cybercrime costs will grow by 15 percent per year over the next five years, reaching \$10.5 trillion USD annually by 2025, up from \$3 trillion USD in 2015 [2]. This surge marks the greatest transfer of economic wealth in history, threatening innovation and investment, exceeding annual damages from natural disasters, and surpassing the profits of the global illegal drug trade.

Effective NID systems are essential in mitigating these risks by providing early detection of intrusions and facilitating rapid response [3]. With the average time to identify and contain a breach estimated at 277 days, early detection is crucial in reducing both the duration and impact of security incidents [4]. Consequently, the role of effective NID systems

is increasingly vital in safeguarding sensitive data, maintaining business continuity, and protecting organisational assets from cyber threats.

In tackling these threats, network intrusion detection primarily relies on two methodologies: signature-based detection and machine learning-based detection [5]. Signature-based detection remains widely adopted due to its high accuracy in identifying known threats by matching network activity against a predefined database of malicious signatures [6]. However, this method has several notable drawbacks:

- 1) Reliance on known patterns of malicious activity, making them ineffective against zero-day attacks and emerging threats that have not yet been catalogued.
- 2) High maintenance requirements arise because signature databases need frequent updates to remain effective against evolving attack patterns, making the process both resource-intensive and time-consuming.
- 3) Performance issues with large signature databases arise as the database of known signatures expands over time, leading to detection delays and negatively impacting overall system efficiency.

Addressing the limitations of signature-based detection methods, machine learning (ML) methods [7], [8] are increasingly favored in network intrusion detection systems. Unlike the classical methods that require constant updates, ML-based detection can analyze vast amounts of network data to identify complex and nuanced patterns associated with both known and previously unknown malicious behavior [9]. Additionally, this adaptability is particularly advantageous given the rapidly changing nature of cyber threats, as seen in cases like the NotPetya attack [10], where ML systems were able to detect anomalous patterns in data traffic before the attack was fully understood [11].

Traditional machine learning (ML) methods, however, still face certain limitations as they rely heavily depend on manual processes, including feature engineering; feature extraction, and selection [12], [13] as shown in Fig. 1. These processes, along with tasks like model selection and tuning add further complexity, making it challenging to handle large-scale intrusion data efficiently. As a result, these techniques often suffer from reduced recognition accuracy and higher false alarm rates.

To overcome the limitations of traditional machine learning, Automated Machine Learning (AutoML) approaches [15] have emerged as powerful alternatives by automating the ML workflow, from data preprocessing to model val-

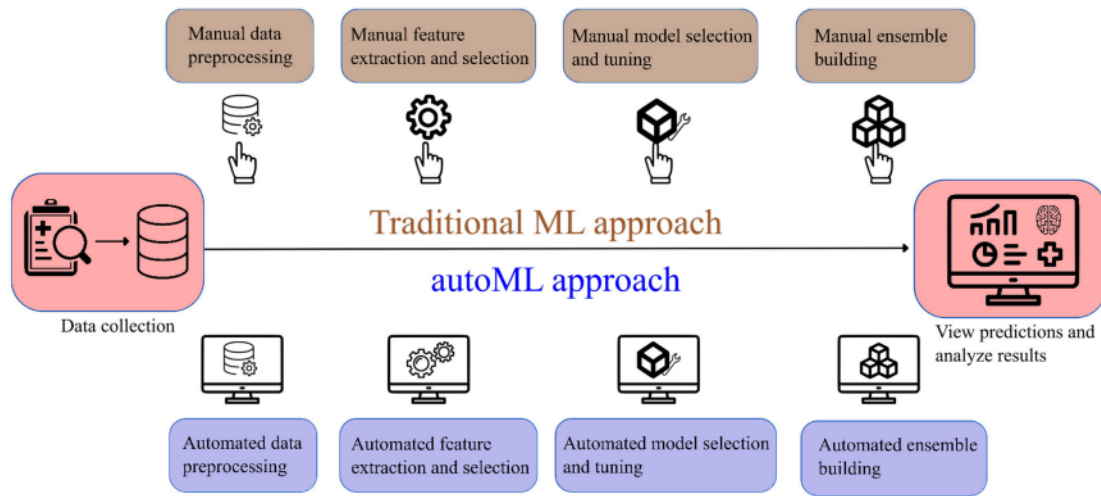


Fig. 1. A comparative overview of traditional ML and AutoML approaches. Traditional ML requires manual steps like data preprocessing, feature extraction, model selection, and tuning, while AutoML automates these tasks, streamlining the workflow from data collection to predictions [14].

identification [16], as shown in Fig. 1 eliminating the need for manual feature selection, enhancing performance, reducing false positives, and optimizing model development. In this paper, we adopt and present a comparative study of AutoML frameworks to develop AutoML-based intrusion detection algorithms that improve recognition accuracy and reduce false alarm rates. The key contributions of this work are as follows:

- 1) Development of AutoML-based prediction algorithms for network intrusion detection to enhance detection accuracy, minimize false positives, and improve overall system robustness.
- 2) A comparative study on the adaptation of conventional ML methods and AutoML-based prediction is undertaken to evaluate their effectiveness in addressing the network intrusion detection tasks.
- 3) Extensive experiments on the NSL-KDD dataset demonstrate the effectiveness of the developed AutoML-based algorithms in boosting detection accuracy and reducing false alarm rates, outperforming traditional ML-based approaches.

The rest of this paper is structured as follows: Section II provides an overview of related research on ML-based intrusion detection approaches. Section III outlines the methodology adopted in this study. Section IV covers the dataset, experimental setup, hyperparameter configurations, and evaluation metrics. Section V discusses the experimental results and insights gained. Finally, Section VI concludes the paper and outlines potential future research directions.

II. RELATED WORKS

This section reviews relevant literature on ML-based intrusion detection methods. As cyber threats evolve, traditional signature-based techniques often fail to detect new attacks. Machine learning (ML) offers a more adaptive solution by analyzing network traffic patterns to identify anomalies. This

review highlights recent advancements and their impact on network intrusion detection.

In [17], a network anomaly detection system was proposed for remote desktop connections using machine learning algorithms. An exponential random graph model was introduced in [18] to integrate network topology statistics for accurate anomaly detection in network domains. In [19], an SDN-based flow prediction method using a double P-value with the K-NN algorithm was proposed, demonstrating higher precision, lower false positives, and significantly better adaptation than other models. Similarly, in [20], a linear kernel-based SVM model was used for anomaly detection in traffic profiles, employing natural language processing techniques to preprocess data and extract feature vectors before classification. In [21], an anomaly detection model extending PCA with group-wise PCA and additional exploratory features was developed based on recent field applications. Additionally, a novel lightweight framework [22] was developed to predict anomalies and cyber-attacks in IoT networks, with evaluations on the NSL-KDD and CICIDS2017 datasets showing improved detection rates and lower false alarm rates. Lastly, [23] proposed an active anomaly detection approach based on deviation- and density-aware features to address abnormal traffic, with a focus on enhancing data privacy.

III. PROPOSED METHODOLOGY

The proposed methodology establishes AutoML as a superior alternative for network intrusion detection through a comparative review of three frameworks: Fast and Lightweight AutoML (FLAML), H2O, and MLJar. This study systematically evaluates their effectiveness in improving detection accuracy. As illustrated in Fig. 1, AutoML streamlines the machine learning pipeline by reducing manual intervention, enhancing detection efficiency, and minimizing false positives, ensuring robust and scalable intrusion detection. The analysis highlights their strengths and

limitations, offering insights into the most effective AutoML approach for cybersecurity applications.

A. AutoML Frameworks for Network Intrusion Detection

This study evaluates three AutoML frameworks, FLAML, H2O, and MLJar, to develop robust network intrusion detection models, as detailed below. These frameworks automate model selection, hyperparameter tuning, and feature engineering, thus streamlining the machine learning pipeline.

1) *FLAML*[24]: This is an efficient AutoML framework that automates model selection, hyperparameter tuning, and feature engineering while minimizing computational cost, making it well-suited for network intrusion detection. Unlike traditional AutoML frameworks that rely on exhaustive search and meta-learning, FLAML employs a cost-aware search strategy that balances accuracy and efficiency. It follows a two-layered system as shown in Fig. 2 where an AutoML layer controls optimization, and an ML layer houses candidate models. The search process selects a resampling strategy, prioritizes learners via the Estimated Cost for Improvement (ECI) metric, and optimizes hyperparameters and sample sizes using randomized direct search. This adaptive mechanism increases model complexity only when necessary, ensuring efficient resource utilization. FLAML's cost-effective trial prioritization, scalability to large datasets, and elimination of meta-learning make it ideal for real-time cybersecurity, enabling efficient and scalable network intrusion detection that outperforms other AutoML frameworks under constrained budgets.

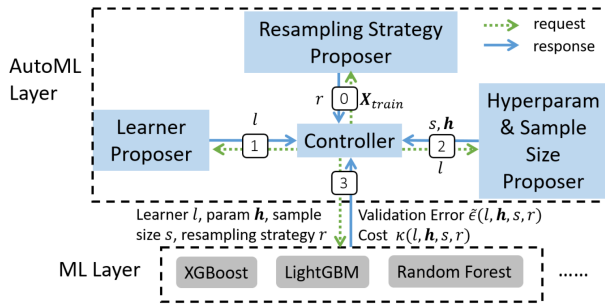


Fig. 2. FLAML's AutoML framework includes an AutoML layer for proposing learners, hyperparameters, and resampling strategies, and an ML layer coordinated by a controller to optimize validation error and computational cost [24].

2) *H2O AutoML*[25]: This is a scalable and fully automated machine learning framework that optimizes model selection, hyperparameter tuning, and ensemble learning for large-scale datasets. It employs random search and stacked ensembles to efficiently explore the model space while maintaining high accuracy. It supports a diverse set of algorithms, including Gradient Boosting Machines (GBMs), Random Forests, Deep Neural Networks, and Generalized Linear Models (GLMs), ensuring robust model diversity. The search process consists of training base models, ranking them on

a leaderboard, and refining predictions using stacked ensembles. Designed for distributed computing, H2O AutoML leverages parallel execution and GPU acceleration, enabling efficient scalability to datasets with millions of instances.

3) *MLJar AutoML* [26]: This is a versatile AutoML framework designed for tabular data, providing four distinct operational modes: "explain" for data analysis and interpretability, "compete" for time-constrained model training, "perform" for production-ready pipelines, and "optuna" for advanced hyperparameter tuning via the Optuna framework. It supports a diverse set of machine learning algorithms enabling robust model selection and performance optimization. Additionally, MLJAR automates feature preprocessing, including missing value imputation and categorical encoding as shown in Fig. 1, while optimizing hyperparameters through random search, Optuna, and hill-climbing techniques. MLJAR facilitates the design and deployment of ensemble models while enhancing model explainability through markdown reports, feature importance analyses, and dependence and decision plots.

B. Machine Learning Algorithms for Network Intrusion Detection

This study examines four machine learning algorithms, Random Forest, XGBoost, CatBoost, and LightGBM, for network intrusion detection, as detailed below. These algorithms leverage ensemble learning and gradient boosting techniques to enhance detection accuracy, model stability, and adaptability to evolving threats.

1) **Random Forest**: This is an ensemble learning method that improves predictive accuracy by combining multiple decision trees. It constructs a "forest" of trees by selecting random subsets of features and data samples, mitigating overfitting and variance. At each node, the algorithm evaluates a subset of features to find the best split, minimizing an impurity measure like Gini impurity (Eq. 1):

$$G_{\text{impurity}} = 1 - \sum_{i=1}^c p_i^2, \quad (1)$$

where p_i is the proportion of class i in the node, and c is the number of classes.

Final predictions for instance x_i are made by aggregating the predictions of all trees via majority voting (Eq. 2):

$$\hat{y}_i = \text{majority_vote}(T_1(x_i), T_2(x_i), \dots, T_K(x_i)), \quad (2)$$

Random Forest is effective due to its robustness to overfitting, ability to handle high-dimensional data, and interpretability via feature importance analysis.

2) **XGBoost (Extreme Gradient Boosting)**: This implementation of the gradient boosting algorithm is both efficient and scalable, featuring regularization, handling of missing data, and parallel computation. The model iteratively builds an ensemble of decision trees, minimizing a regularized objective function (Eq. 3):

$$\mathcal{L}(\theta) = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k), \quad (3)$$

where $l(y_i, \hat{y}_i)$ is the loss function and $\Omega(f_k)$ penalizes model complexity.

At each iteration, a new tree corrects residuals, updating predictions:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i)$$

XGBoost leverages first-order gradients g_i and second-order Hessians h_i for precise optimization. The regularization term (Eq. 4) helps control overfitting:

$$\Omega(f_k) = \gamma T + \frac{1}{2} \lambda \|\omega\|^2, \quad (4)$$

This, combined with parallelization, makes XGBoost highly effective for large-scale predictive modeling tasks.

3) **CatBoost**: This is a gradient boosting algorithm optimized for handling categorical features and designed for high efficiency in both accuracy and speed. Unlike traditional methods, it uses innovative encoding techniques for categorical variables, ideal for datasets with high cardinality. The regularized objective function (Eq. 5) is minimized as follows:

$$\mathcal{L}(\theta) = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k), \quad (5)$$

where $l(y_i, \hat{y}_i)$ is the loss function, and $\Omega(f_k)$ regularizes model complexity.

At each iteration, a new tree minimizes residuals: $\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i)$. CatBoost uses Ordered Boosting to reduce prediction shift and Symmetric Trees to improve both accuracy and speed. The regularization term (Eq. 6) helps control overfitting:

$$\Omega(f_k) = \lambda \|\omega\|^2 + \alpha T, \quad (6)$$

These features, combined with gradient and second-order gradient information, make CatBoost highly effective for large-scale, high-dimensional datasets with categorical variables.

4) **LightGBM(LGBM)**: This is a highly efficient gradient boosting implementation, designed for improved speed and scalability. Unlike traditional methods, LGBM uses a leaf-wise tree growth strategy, splitting the leaf with the maximum loss reduction, which results in deeper trees and higher accuracy. It is well-suited for large datasets due to its ability to handle sparse features, reduce memory usage, and support parallel learning.

The objective function minimizes a regularized loss, as shown in Eq. 7:

$$\mathcal{L}(\theta) = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k), \quad (7)$$

where $l(y_i, \hat{y}_i)$ is the loss function and $\Omega(f_k)$ controls model complexity.

At each iteration, LGBM adds a new tree based on leaf-wise splits, updating predictions:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i)$$

LGBM uses first- and second-order gradients for efficient optimization. The regularization function (Eq. 8) is:

$$\Omega(f_k) = \gamma T + \frac{1}{2} \lambda \|\omega\|^2, \quad (8)$$

This controls overfitting while ensuring performance. With its ability to handle large data and provide accurate predictions, LGBM is widely adopted for tasks requiring speed and memory efficiency.

IV. DATASET, EXPERIMENT SETUP, HYPER-PARAMETER TUNING AND EVALUATION METRICS

This section describes the dataset, experimental setup, hyperparameter tuning, and performance evaluation metrics used in the network intrusion detection models.

A. Dataset

The dataset used in this study is the NSL-KDD dataset [27], a refined version of the widely recognized KDD Cup 1999 dataset [28]. The NSL-KDD dataset is a popular benchmark for evaluating network intrusion detection (NID) systems. Unlike its predecessor, the NSL-KDD addresses issues with redundant records and provides a more balanced distribution of data across both the training and testing sets, which improves the fairness and robustness of system evaluations. Specifically, it includes 125,973 records for training (NSL-KDDTrain+) and 22,544 records for testing (NSL-KDDTest+), ensuring comprehensive testing across diverse traffic patterns as shown in Table I.

TABLE I

Distribution of classes in the NSL-KDD training and testing sets.

	normal	Dos	Probe	U2R	R2L	Total
Train	67,343	45,927	11,656	52	995	125,973
Test	9,711	7,458	2,421	200	2,754	22,543

The dataset consists of five classes: Normal, Denial of Service (DoS), User to Root (U2R), Remote to Local (R2L), and Probe, each representing different types of network traffic as shown in Table I. Normal corresponds to legitimate traffic, while the four attack classes reflect various malicious activities. The dataset includes 41 features, divided into three categories: basic features (1–10), such as protocol type and service; content features (11–22), such as the number of failed login attempts; and traffic features (23–41), which include the number of connections to the same host within a given time window [27].

B. Hyper-parameter Tuning

The hyperparameters of the classifiers used in the NID models are optimized using Bayesian optimization with 10-fold cross-validation. In this process, the dataset is split into 10 equally sized subsets (folds), with each algorithm trained on 9 folds and validated on the remaining one. This is repeated 10 times, allowing each fold to serve as the validation set once. The average performance across all folds ensures robust model evaluation, minimizing reliance on any specific train-test split. The data is shuffled before splitting to diversify training and validation sets, reducing overfitting and improving generalization. Stratification is applied to maintain class proportions in each fold. The best-performing configuration is selected based on average performance across the folds, as summarized in Table II.

TABLE II
Classifiers Hyper-parameter Settings.

Classifier	Hyper-parameter	Values
Random Forest	Criterion	Gini
	max_features	0.5
	min_samples_split	20
	max_depth	4
XGBoost	learning_rate (eta)	0.075
	max_depth	8
	min_child_weight	5
	subsample	1.0
	colsample_bytree	1.0
CatBoost	learning_rate	0.05
	depth	8
	rsm	0.8
	num_leaves	63
LightGBM	learning_rate	0.05
	feature_fraction	0.9
	bagging_fraction	0.9
	min_data_in_leaf	5

C. Evaluation Metrics

The weighted average values of Precision, Recall, and F1-score, as defined in Eqs. 10 through 12, along with accuracy as defined in Eq. 9, are adopted to evaluate the performance of the NID models [29]. These metrics are calculated using the true positive (tp_i), true negative (tn_i), false positive (fp_i), and false negative (fn_i) values for each class C_i , where $i = 1, \dots, m$ and m represents the total number of classes in the dataset. Here, $|Y_i|$ denotes the total number of samples assigned to each class.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (9)$$

$$\text{Weighted Average Precision} = \frac{\sum_{i=1}^m |Y_i| \frac{tp_i}{tp_i + fp_i}}{\sum_i |Y_i|} \quad (10)$$

$$\text{Weighted Average Recall} = \frac{\sum_{i=1}^m |Y_i| \frac{tp_i}{tp_i + fn_i}}{\sum_i |Y_i|} \quad (11)$$

$$\text{Weighted Average F1-score} = \frac{\sum_{i=1}^m |Y_i| \frac{2tp_i}{2tp_i + fp_i + fn_i}}{\sum_i |Y_i|} \quad (12)$$

D. Experiment Setup

The dataset used for the experiments is described in section IV-A. The training set (NSL-KDDTrain+) which includes 125,973 records is used to train the intrusion detection models and the testing dataset (NSL-KDDTest+) which includes 22,544 records is used to evaluate the detection models. For this study, the dataset has been transformed into binary classification tasks, distinguishing between Normal and Attack (malicious) traffic where the the four attack classes reflect various malicious activities.

To evaluate the performance of the proposed framework, Random Forest (RF), Xgboost, Catboost, lightgbm and the adopted AutoML framework classifiers are used to evaluate the performance of the NID models. An *NVIDIA – SMI A100 – SXM4-40GB* GPU on *GoogleColab+* is used in training the models.

V. RESULTS AND DISCUSSION

Table III demonstrate the performance metrics; Accuracy, Precision, Recall, and *F1 Score* of different machine learning models, including Random Forest, XGBoost, CatBoost, and LightGBM, as well as the final Stacked Ensemble model, for network intrusion detection.

TABLE III
Performance metrics for evaluated models, with precision, recall, and *F1 – score* reported as weighted averages.

Model	Accuracy	Precision	Recall	F1 Score
Random Forest	78	84	78	78
XGBoost	80	85	80	80
CatBoost	80	85	80	80
LGBM	78	84	78	78
FLAML	79	84	79	79
H ₂ O	90	91	90	90
MLJar	90	90	89	89

Among the traditional machine learning models, XGBoost and CatBoost achieved the highest performance, both attaining 80% accuracy, 85% precision, 80% recall, and an F1-score of 80% (Table III). These results suggest that gradient boosting techniques effectively enhance detection accuracy by optimizing decision trees in an ensemble manner. Random Forest and LightGBM, while still performing well, yielded slightly lower accuracy (78%) and F1-score (78%), indicating that bagging techniques may be less effective than boosting in this context. Figure 3 further illustrates these performance differences, highlighting variations across accuracy, precision, recall, and F1-score.

The AutoML frameworks demonstrated superior performance, with H2O AutoML and MLJar achieving the highest overall accuracy (90%). H2O AutoML marginally outperformed MLJar, showing a precision of 91%, recall of 90%, and an F1-score of 90%, while MLJar attained 90% precision, 89% recall, and an F1-score of 89% (Table III). These results indicate that fully automated model selection, hyperparameter tuning, and ensemble optimization significantly enhance detection capabilities. Figure 4 further illus-

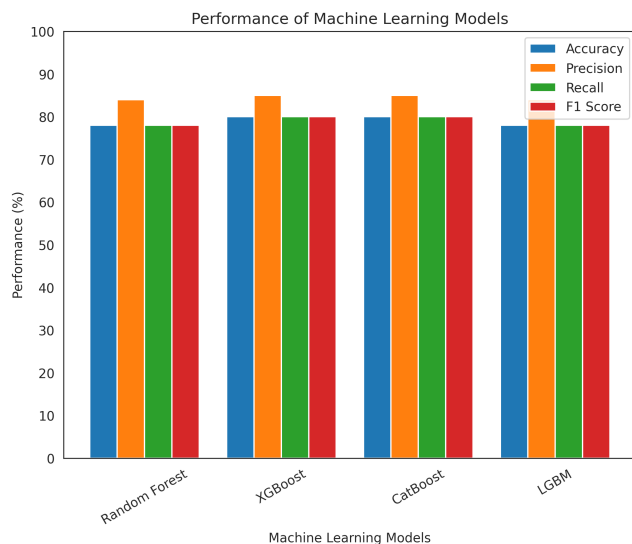


Fig. 3. Performance comparison of machine learning models for network intrusion detection, highlighting the superiority of gradient boosting (XGBoost, CatBoost) over bagging-based methods (Random Forest, LightGBM).

trates these findings, highlighting the advantages of AutoML frameworks over traditional models.

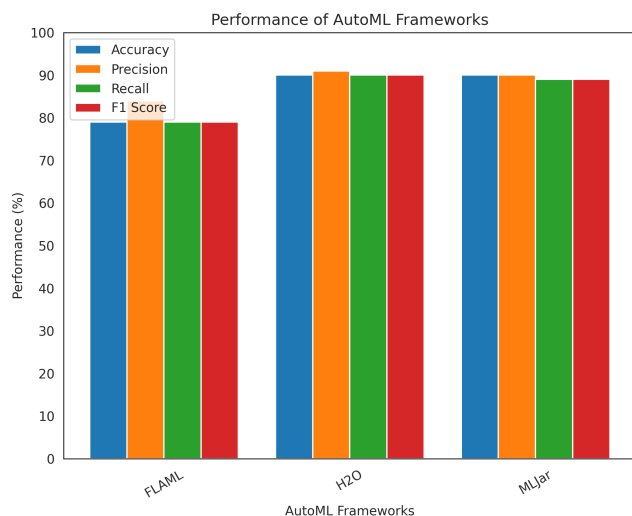


Fig. 4. Performance comparison of AutoML frameworks (FLAML, H2O, and MLJar) in network intrusion detection, based on accuracy, precision, recall, and F1-score. H2O and MLJar achieve higher performance across all metrics, highlighting the impact of advanced hyperparameter tuning and ensemble learning.

While FLAML improves upon traditional machine learning models, it underperforms compared to H2O AutoML and MLJar, achieving 79% accuracy, 84% precision, 79% recall, and an F1-score of 79%. Its lower performance is due to FLAML's lightweight, cost-aware design, which prioritizes computational efficiency over exhaustive model exploration. Unlike H2O AutoML and MLJar, which leverage comprehensive hyperparameter tuning, stacked ensembles, and deep model optimization, FLAML focuses on fast, low-cost model selection with minimal resource consumption. While this

makes FLAML advantageous for real-time applications, it limits its ability to achieve peak detection performance. Additionally, FLAML does not incorporate advanced ensemble learning techniques, such as multi-layer stacking and blending, which contribute to the superior performance of H2O AutoML and MLJar.

Overall, these results emphasize the effectiveness of AutoML frameworks in optimizing network intrusion detection. The superior performance of H2O AutoML and MLJar suggests that automated hyperparameter tuning, model stacking, and feature engineering provide a competitive edge over manually optimized models. Thus, integrating AutoML frameworks into network intrusion detection not only enhances detection accuracy but also streamlines model development, making intrusion detection systems more adaptive and scalable.

VI. CONCLUSION AND FUTURE WORK

In this paper, we evaluated AutoML frameworks (FLAML, H2O, and MLJar) alongside traditional machine learning models for network intrusion detection. The results show that while gradient boosting models outperform bagging-based methods, AutoML frameworks achieve superior accuracy and efficiency through automated model selection, hyperparameter tuning, and ensemble learning. Among them, H2O and MLJar (90% accuracy) outperformed FLAML, which, despite its computational efficiency, exhibited slightly lower detection accuracy due to its lightweight design. These findings highlight AutoML's advantages in enhancing intrusion detection, improving detection robustness while reducing the need for manual tuning, making it a valuable approach for strengthening cybersecurity systems.

In our future work, we will focus on evaluating AutoML-based models on more recent, real-world network traffic datasets to improve generalizability and address the limitations of the NSL-KDD dataset. Additionally, we plan to explore the scalability and real-time performance of these models in high-throughput cybersecurity environments to assess their practical deployment. We will also investigate the interpretability of AutoML-generated models to ensure their usability in security operations. Finally, we aim to analyze AutoML's adaptability to evolving cyber threats, providing deeper insights into its long-term effectiveness in dynamic security landscapes.

ACKNOWLEDGEMENT

This work is supported by the National Center for Transportation Cybersecurity and Resiliency (TraCR) (a U.S. Department of Transportation National University Transportation Center) headquartered at Clemson University, Clemson, South Carolina, USA. Any opinions, findings, conclusions, and recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of TraCR. The U.S. Government assumes no liability for the contents or use thereof.

REFERENCES

- [1] N. Chaabouni, M. Mosbah, A. Zemmari, C. Sauvignac, and P. Faruki, "Network intrusion detection for iot security based on learning techniques," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2671–2701, 2019.
- [2] cybercrimemag, "Cybercrime To Cost The World \$10.5 Trillion Annually By 2025 — cybersecurityventures.com," <https://cybersecurityventures.com/hackerpocalypse-cybercrime-report-2016/>, [Accessed 12-10-2024].
- [3] N. Rawindaran, A. Jayal, E. Prakash, and C. Hewage, "Cost benefits of using machine learning features in nids for cyber security in uk small medium enterprises (sme)," *Future Internet*, vol. 13, no. 8, p. 186, 2021.
- [4] "Cost of a data breach 2024 | IBM — ibm.com," <https://www.ibm.com/reports/data-breach>, [Accessed 12-10-2024].
- [5] C. Zhang, D. Jia, L. Wang, W. Wang, F. Liu, and A. Yang, "Comparative research on network intrusion detection methods based on machine learning," *Computers & Security*, vol. 121, p. 102861, 2022.
- [6] P. Ioulianou, V. Vasilakis, I. Moscholios, and M. Logothetis, "A signature-based intrusion detection system for the internet of things," *Information and Communication Technology Form*, 2018.
- [7] M. Panda, A. Abraham, S. Das, and M. R. Patra, "Network intrusion detection system: A machine learning approach," *Intelligent Decision Technologies*, vol. 5, no. 4, pp. 347–356, 2011.
- [8] W. Li, P. Yi, Y. Wu, L. Pan, and J. Li, "A new intrusion detection system based on knn classification algorithm in wireless sensor network," *Journal of Electrical and Computer Engineering*, vol. 2014, no. 1, p. 240217, 2014.
- [9] M. S. Habeeb and T. R. Babu, "Network intrusion detection system: a survey on artificial intelligence-based techniques," *Expert Systems*, vol. 39, no. 9, p. e13066, 2022.
- [10] A. Greenberg, "The Untold Story of NotPetya, the Most Devastating Cyberattack in History — wired.com," <https://www.wired.com/story/notpetya-cyberattack-ukraine-russia-code-crashed-the-world/>, [Accessed 12-10-2024].
- [11] A. H. Salem, S. M. Azzam, O. Emam, and A. A. Abohany, "Advancing cybersecurity: a comprehensive review of ai-driven detection techniques," *Journal of Big Data*, vol. 11, no. 1, p. 105, 2024.
- [12] S. K. Wagh, V. K. Pachghare, and S. R. Kolhe, "Survey on intrusion detection system using machine learning techniques," *International Journal of Computer Applications*, vol. 78, no. 16, pp. 30–37, 2013.
- [13] S. Garg and S. Batra, "A novel ensembled technique for anomaly detection," *International Journal of Communication Systems*, vol. 30, no. 11, p. e3248, 2017.
- [14] R. Raj, S. K. Kannath, J. Mathew, and P. Sylaja, "Automl accurately predicts endovascular mechanical thrombectomy in acute large vessel ischemic stroke," *Frontiers in Neurology*, vol. 14, p. 1259958, 2023.
- [15] I. Salehin, M. S. Islam, P. Saha, S. Noman, A. Tunj, M. M. Hasan, and M. A. Baten, "Automl: A systematic review on automated machine learning with neural architecture search," *Journal of Information and Intelligence*, vol. 2, no. 1, pp. 52–81, 2024.
- [16] F. Hutter, L. Kotthoff, and J. Vanschoren, *Automated machine learning: methods, systems, challenges*. Springer Nature, 2019.
- [17] R. Bitton and A. Shabtai, "A machine learning-based intrusion detection system for securing remote desktop connections to electronic flight bag servers," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 3, pp. 1164–1181, 2019.
- [18] M. Tsikerdekis, S. Waldron, and A. Emanuelson, "Network anomaly detection using exponential random graph models and autoregressive moving average," *IEEE Access*, vol. 9, pp. 134 530–134 542, 2021.
- [19] H. Peng, Z. Sun, X. Zhao, S. Tan, and Z. Sun, "A detection method for anomaly flow in software defined network," *IEEE Access*, vol. 6, pp. 27 809–27 817, 2018.
- [20] Q. Ma, C. Sun, B. Cui, and X. Jin, "A novel model for anomaly detection in network traffic based on kernel support vector machine," *Computers & Security*, vol. 104, p. 102215, 2021.
- [21] J. Camacho, R. Therón, J. M. García-Giménez, G. Maciá-Fernández, and P. García-Teodoro, "Group-wise principal component analysis for exploratory intrusion detection," *IEEE Access*, vol. 7, pp. 113 081–113 093, 2019.
- [22] S. Roy, J. Li, B.-J. Choi, and Y. Bai, "A lightweight supervised intrusion detection mechanism for iot networks," *Future Generation Computer Systems*, vol. 127, pp. 276–285, 2022.
- [23] B. Li, Y. Wang, K. Xu, L. Cheng, and Z. Qin, "Dfraid: Density-aware and feature-deviated active intrusion detection over network traffic streams," *Computers & Security*, vol. 118, p. 102719, 2022.
- [24] C. Wang, Q. Wu, M. Weimer, and E. Zhu, "Flaml: A fast and lightweight automl library," *Proceedings of Machine Learning and Systems*, vol. 3, pp. 434–447, 2021.
- [25] E. LeDell and S. Poirier, "H2o automl: Scalable automatic machine learning," in *Proceedings of the AutoML Workshop at ICML*, vol. 2020, 2020, p. 24.
- [26] A. Płońska and P. Płoński, "Mljar: State-of-the-art automated machine learning framework for tabular data. version 0.10.3," Łapy, Poland, 2021. [Online]. Available: <https://github.com/mljar/mljar-supervised>
- [27] L. Dhanabal and S. Shantharajah, "A study on nsl-kdd dataset for intrusion detection system based on classification algorithms," *International journal of advanced research in computer and communication engineering*, vol. 4, no. 6, pp. 446–452, 2015.
- [28] V. Engen, J. Vincent, and K. Phalp, "Exploring discrepancies in findings obtained with the kdd cup'99 data set," *Intelligent Data Analysis*, vol. 15, no. 2, pp. 251–276, 2011.
- [29] M. Al Lail, A. Garcia, and S. Olivo, "Machine learning for network intrusion detection—a comparative study," *Future Internet*, vol. 15, no. 7, p. 243, 2023.